

## Assignment 2 (20% of total marks)

**Due date: Saturday, 14 February 2026 by 9:00 pm (21:00 hours)  
Singapore time.**

### Scope

The tasks of this assignment cover the **data structure and algorithm**. Focusing on data structure and algorithm design. The assignment covers the topics discussed in **Topic 4**, and **Topic 5**.

The assignment consists of:

- **Part A:** Theory questions to assess your understanding of the subject matter and support the programming question's objective.
- **Part B:** Programming questions requiring the implementation of concepts related to algorithms and their complexity.

Marks for each question are indicated next to the respective question. The total mark for Assignment 2 is **100 marks**.

### Marks Distribution

- **Part A:** 70%
- **Part B:** 30%
- **Total Marks:** 100
- **Weightage:** 15% of the total subject mark

### Assessment Criteria

Marks will be awarded for:

- **Correctness:** Solutions must meet the requirements of the question.
- **Comprehensiveness:** Solutions should demonstrate a complete understanding of the topic.
- **Appropriate Application:** Answers should reflect proper application of the materials covered in this subject.

# Deliverables

## Part A: Theory Questions (Questions 1 to 3)

- Type your answers using **MS Word** and save the completed document as a **PDF file**.
- Alternatively, you may write your answers by hand onto a piece of paper or using tablet. Ensure your handwriting is **neat**, then scan or save your handwritten solutions into a **PDF file**.

## Part B: Programming Question (Question 4)

- Implement your solution following the standard requirements provided for the question. You may use **C, C++, Java, or Python**.
- Execute your program and take a **screenshot of the output**.
- Save your source code in **pure ASCII text format** with the appropriate file extension (e.g., **.cpp** for C++, **.java** for Java, or **.py** for Python).
- Ensure your program is appropriately documented with comments.
- Provide clear **compilation instructions** in a readme.txt file. Include batch or makefiles if applicable for C or C++ programs.
  - *Note:* If no compilation instructions are provided, your lecturer will attempt compilation based on their environment. Any failure due to incompatibility will result in a "failure to compile" outcome.

## Submission Instructions

- Combine the following into a single **ZIP file** named **SolutionA2.zip**:
  - Your solutions to the theory questions (in **PDF format**).
  - Your source code for the programming question (in **ASCII text format**).
  - The screenshot of your program's output.
- Do not include subdirectories in your submission.

## Part A: Theory Questions (70 Marks)

### Question 1 – Binary Tree (15 Marks)

- a) The INORDER traversal output of a binary tree is H,Y,P,H,E,N,A,T,I,O,N and the PREORDER traversal output of the same tree is H,O,P,Y,T,E,H,A,N,I,N. Construct the tree and determine the output of the POSTORDER traversal output.

**(5.0 marks)**

- b) One main difference between a binary search tree (BST) and an AVL (Adelson-Velski and Landis) tree is that an AVL tree has a balance condition, that is, for every node in the AVL tree, the height of the left and right subtrees differ by at most 1. Starting with an empty BST and AVL tree, insert the following values into the two trees (BST as well as AVL trees), in other words, first insert the values and construct a BST, then using the same set of the values insert and construct an AVL tree.

#### Tasks:

- i. Starting with an empty BST (Binary Search Tree), insert the following values into the BST tree.

25, 36, 14, 18, 29, 26, 48, 38, 43

If a key already existed, insert the new key to the LEFT sub-tree.

**(2.0 marks)**

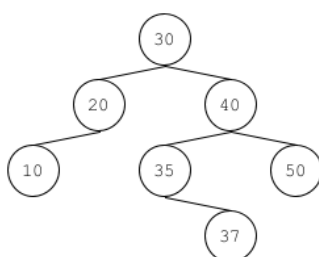
- ii. Starting with an empty AVL (Balanced Binary Search Tree), insert the following values into the AVL tree.

25, 36, 14, 18, 29, 26, 48, 38, 43

If a key already existed, insert the new key to the LEFT sub-tree. Show the rebalancing steps and rotations.

**(3.0 marks)**

- c) Start with this initial BST (Binary Search Tree) tree:

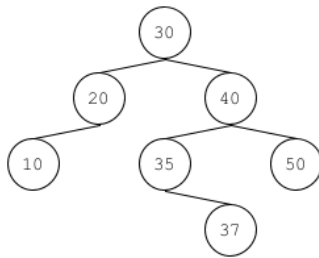


#### Task:

Delete 30 from the BST tree and draw the resulting tree. Explain the process of deletion.

**(2.0 marks)**

- d) Start with this initial AVL (Balanced Binary Search Tree) tree:



**Task:**

Delete 30 from the AVL tree. Show the rebalancing steps and rotations. Draw the AVL after the deletion.

**(3.0 marks)**

## Question 2 – Queue and Stack (15 Marks)

Two algorithms claim to return the same value from a binary tree.

```
Function B1(root)
    if (root == NULL) return -1
    x = root.data
    Q = new Queue()
    ENQUEUE(Q, root)

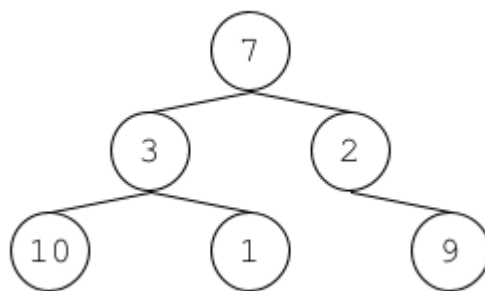
    while (!ISEMPTY(Q)) do
        t = DEQUEUE(Q)
        if (t.data > x) then
            x = t.data
        endif
        ENQUEUE(Q, t.left)
        ENQUEUE(Q, t.right)
    endwhile

    return x
EndFunction
```

```
Function B2(root)
    if (root == NULL) return -1
    t = root
    while (t.right != NULL) do
        t = t.right
    endwhile
    return t.data
EndFunction
```

Note: the function ENQUEUE only inserts a new element in the queue if this element is not NULL. In other words, ENQUEUE ignores NULL.

a) For the following Binary Tree:



- i. What is returned by B2(root)? **(2.5 marks)**
- ii. What is returned by B1(root)? **(2.5 marks)**

- b) For the Binary Tree in part (a), what is the content of the queue immediately before returning from the execution of function B1(root)? **(5.0 marks)**
- c) Re-write the function B2, in pseudocode, using recursive function calls. You **may not** use any form of iteration. **(5.0 marks)**

### Question 3 – Associative Table (20 Marks)

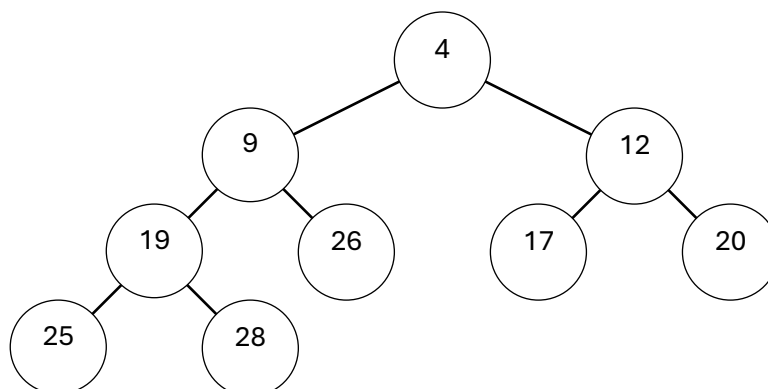
Consider a hash table of size 13 with hash function  $h(x) = 3x \bmod 13$ . Draw the table that results after inserting, in the given order, the following values: 18, 41, 22, 44, 59, 32, 31, 73, 14 for each of the three scenarios below:

- a) When collisions are handled by separate chaining; **(5.0 marks)**
- b) When collisions are handled by linear probing; **(5.0 marks)**
- c) When collisions are handled by double hashing using a second hash function  $h_2(x) = 13 - (x \bmod 13)$ . Hint, the overall (combined) hash function is  $H(x) = (h(x) + i \times h_2(x)) \bmod 13$ , where  $i = 0, 1, 2, 3, \dots$  **(5.0 marks)**
- d) When collisions are handled by quadratic probing with a quadratic probe function  $h_3(x, i) = (h(x) + 0.5 i + 0.5 i^2) \bmod 13$  where  $i = 1, 2, 3, \dots$  **(5.0 marks)**

### Question 4 – Heap (20 marks)

- a) Given a binary min-heap  $h$  stored as an array of  $n$  keys, design an algorithm *convertToMaxHeap(h)* that transforms  $h$  **in-place** into a valid binary max-heap. Your algorithm must run in  $O(n \log n)$  time. Provide a brief runtime analysis to justify the bound. **(10.0 marks)**

- a. Given the following minimum heap, illustrate the process of converting it into a maximum heap using your algorithm described in (a). You need to show the intermediate processes. **(10.0 marks)**



## Part B: Programming Question (30 Marks)

### Assignment Brief: Logistics Hub Distribution Simulation

#### 1. Overview

Your task is to investigate the efficiency of a two-stage automated logistics hub. Packages arrive at a **Sorting Station** (Primary) and, once processed, move to a **Loading Dock** (Secondary) for final dispatch. You must simulate this flow to determine bottlenecks and server efficiency.

#### 2. Implementation Constraints

- **Language:** Java, C++, or Python.
- **No Library Queues:** You are **strictly prohibited** from using built-in queue classes (e.g., `java.util.LinkedList`, `std::queue`, or `collections.deque`).
- **Custom Implementation:** You must write your own Queue class/struct using arrays or linked nodes.

#### 3. System Logic

1. **Stage 1 (Sorting):** All packages enter a single "Arrival Queue." When a Sorting Server becomes available, the package is processed.
2. **Stage 2 (Loading):** Immediately after being sorted, the package enters a "Loading Queue." When a Loading Dock Server becomes available, the package is loaded for dispatch.
3. **Non-Blocking Flow:** A package cannot start Loading until its Sorting time is complete.

#### 4. Input Format

The program should read from a data file provided via command line.

- **Line 1:** Two integers: S (Number of Sorting Servers) and L (Number of Loading Dock Servers).
- **Subsequent Lines:** Each line represents one package: Arrival\_Time, Sorting\_Time, and Loading\_Time.
- **Termination:** A dummy record 0 0 0 signals the end of the input.

#### Example Data (logistics.dat):

```
2 1
1 5 10
2 3 5
5 2 2
0 0 0
```

(Translation: 2 Sorting Servers, 1 Loading Dock. Package 1 arrives at minute 1, takes 5 mins to sort and 10 mins to load.)

## 5. Required Output

Upon completion (when the last package has left the Loading Dock), output the following statistics:

- **Throughput:** Total number of packages processed.
- **Final Clock Time:** The time the last package cleared the system.
- **Processing Metrics:**
  - Average stay time in the hub (Arrival to Departure).
  - Average wait time in the Sorting Queue.
  - Average wait time in the Loading Queue.
- **Queue Analysis:**
  - Maximum length reached by the Sorting Queue.
  - Maximum length reached by the Loading Queue.
- **Server Efficiency:**
  - Total idle time for each individual Sorting Server.
  - Total idle time for each individual Loading Dock Server.

## 6. The Test Data

A2data2.txt



## Notes on Submission

1. This assignment is due on **Saturday, 14 February 2026 by 9:00 pm (21:00 hours) Singapore time.**
2. Submission must be made via **Moodle**. Ensure all files are combined into a single **ZIP file** without subdirectories.
3. Clearly label all components of your submission.
4. Late submissions will incur a **5% deduction per day**, including weekends.
5. Submissions more than **seven** days late will not be marked unless an extension is granted.
6. Extensions must be applied for via **SOLS** before the assignment deadline.
7. Plagiarism is treated seriously. Instances of plagiarism may result in a zero mark for the assignment.

Submit the file **SolutionA2.zip** through Moodle in the following way:

- 1) Access Moodle at **<http://moodle.uowplatform.edu.au/>**
- 2) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
- 3) When successfully logged in, select a site **CSCI203 (SP126) Algorithms and Data Structures**
- 4) Scroll down to a section **Submissions of Assignments**
- 5) Click at **Submit** your Assignment 2 here link.
- 6) Click at a button **Add Submission**
- 7) Move the files created into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add...*
- 8) Click at a button **Save** changes,
- 9) Click at check box to confirm authorship of a submission,
- 10) When you are satisfied, remember to click at a button **Submit assignment**.

**A policy regarding late submissions is included in the subject outline. Only one submission per student is accepted.**

Assignment 2 is an individual assignment, and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

---

*End of specification*

If you have any questions or require clarification, please contact your lecturer as early as possible. Good luck!